

# ActionResult<T>

```
[HttpGet("{id:int}")] // api/autores/id
public async Task<ActionResult<Autor>> Get(int id)
{
    var autor = await context.Autores
        .Include(x => x.Libros)
        .FirstOrDefaultAsync(x => x.Id == id);

    if (autor is null)
    {
        return NotFound();
    }

    return autor;
}
```

# IActionResult

```
[HttpGet("{parametro1}/{parametro2?}")]
public IActionResult Get(string parametro1, string parametro2 = "valor por defecto")
{
    return Ok(new { parametro1, parametro2 });
}
```

## Task<T>

```
public async Task<IEnumerable<Autor>> Get()
{
    return await context.Autores.ToListAsync();
}
```

## Task

```
public async Task CrearAutor(Autor autor)
{
    context.Add(autor);
    await context.SaveChangesAsync();
}
```

## ValueTask y ValueTask<T>

- Son tipos de valor
- Los utilizamos cuando tenemos escenarios en los cuales tenemos métodos los cuales a veces pueden ser síncronos y otras veces pueden ser asíncronos

## Tiempo de Vida de Servicios

- Transient
- Scoped
- Singleton

### Middleware

## Middleware

- Una **pipe (tubería)** es una cadena de procesos conectados de tal forma que la salida de cada elemento de la cadena es la entrada del próximo.
- A cada uno de los procesos le llamamos middleware
- Ejemplo: el middleware de autorización



